

1. Author

Name: Dua Saeed

Roll Number: 23f1000825

Email: 23f1000825@ds.study.iitm.ac.in

About Me: I am a Computer Science and Engineering student with a keen interest in web application development, data engineering, and software development. I enjoy designing and building practical, user-centric applications while continuously expanding my knowledge of modern technologies and best development practices.

2. AI/LLM Usage

AI tools used:: ChatGPT (OpenAI)

The tool was used to assist with debugging and improving the user interface. All implementation decisions, testing, and integration were completed manually.

Estimated AI usage: Approximately **10-20%**

3. Description

The objective of this project was to develop a Trek Management System with separate roles for Admin, Trek Staff, and Trekker. Each role has its own dashboard and permissions for managing

4. Technologies Used

Technology	Purpose
Python	Backend programming language
Flask	Web framework
SQLAlchemy	Object Relational Mapper (ORM)
SQLite	Database
Flask-Login	User authentication and session management
Jinja2	HTML template rendering

Bootstrap 5	Responsive user interface
Chart.js	Dashboard charts and data visualization
HTML5 & CSS3	Frontend structure and styling

These technologies were selected because they integrate well with Flask and are suitable for developing a lightweight database-driven web application.

5. Database Schema / ER Diagram

The application uses four main tables:

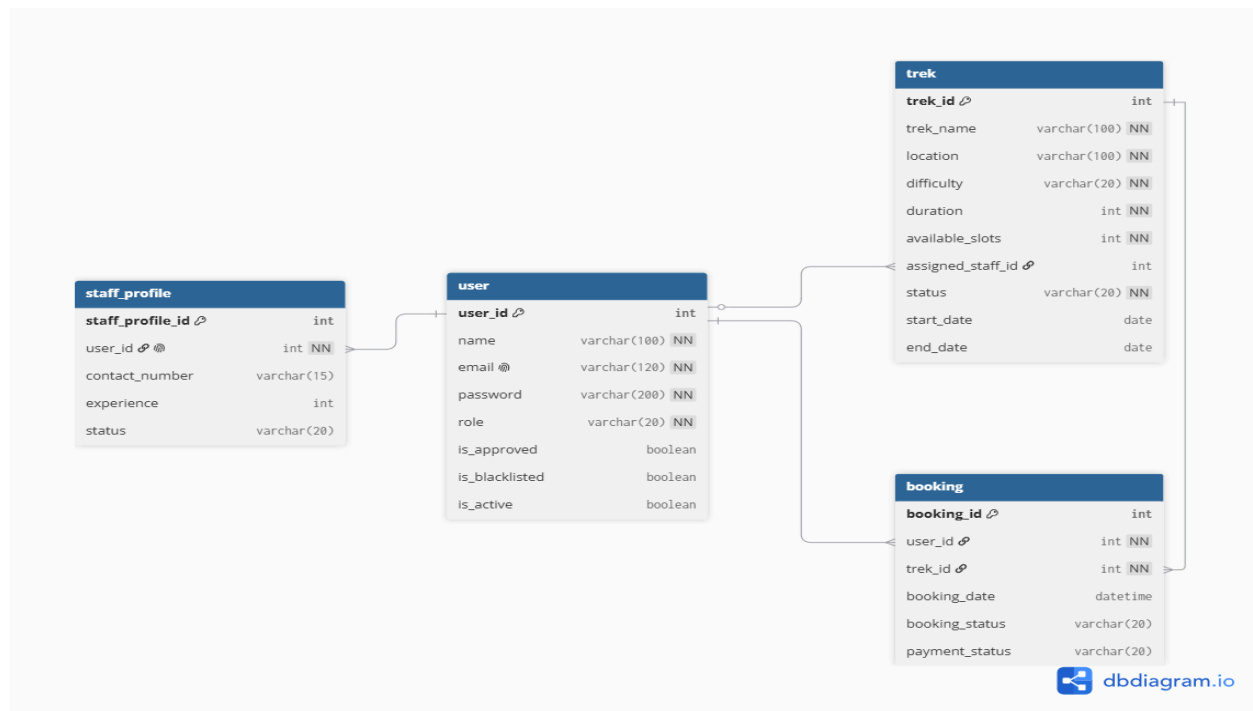
- **User** – Stores Admin, Trek Staff, and Trekker accounts.
- **Trek** – Stores trek information, assigned staff, and current status.
- **Booking** – Stores trek bookings and booking/payment status.
- **StaffProfile** – Stores additional information for Trek Staff.

Relationships

- One User → Many Bookings
- One Trek → Many Bookings
- One Trek Staff → Many Assigned Treks
- One Trek Staff → One Staff Profile

Constraints

- Email addresses are unique.
- Each booking belongs to one user and one trek.
- Duplicate bookings are not allowed.
- Users cannot book closed or full treks.



6. API Design

The project does not include REST API endpoints. All application functionality is implemented using Flask routes that render HTML templates and process form submissions.

The optional API Integration milestone was not implemented. Therefore, no OpenAPI/YAML specification file is included with this submission

7. Architecture and Features

The project is organized using a modular Flask structure. The **main.py** file contains the application setup and route handlers, while **models.py** defines the database models using SQLAlchemy and **database.py** manages the database connection. The user interface is built using Jinja2 templates organized into separate folders for authentication, admin, trek staff, and user pages, with Bootstrap used for responsive styling. The application implements role-based authentication using Flask-Login, allowing Admins to manage treks, staff, users, and bookings, Trek Staff to manage their assigned treks and participants, and Trekkers to browse, search, and book treks. Additional features include dashboard analytics using [Chart.js](#).

8. Video

Google Drive Link: [23f1000825_VideoReport_MAD1.mp4](#)